

Guia do Aluno

Como usar, peça por peça, o Framework MVC Didático em PHP — do primeiro acesso até criar o seu próprio CRUD.

Projeto: Framework MVC — Curso Técnico

Banco de dados: MySQL / MariaDB (XAMPP)

Requisitos: PHP 8.1 ou superior

Para quem é este guia: para quem já sabe o básico de PHP (variáveis, `if`, `foreach`, funções e arrays) e agora vai aprender a organizar um sistema de verdade usando o padrão MVC.

Sumário

1. Colocando o sistema para rodar

Requisitos · Instalação em 4 passos · Conferindo se deu certo

2. O que é MVC (e por que usar)

3. O caminho de uma requisição

4. Mapa das pastas

5. As peças do framework

5.1 index.php · 5.2 bootstrap.php · 5.3 Autoloader · 5.4 Config
5.5 App (roteador) · 5.6 Controller · 5.7 Model · 5.8 View e template
5.9 Validador · 5.10 Sessão e mensagens · 5.11 Database
5.12 Sql (segurança) · 5.13 Funções de atalho

6. Como adicionar cada parte do sistema

6.1 Uma tabela · 6.2 Um modelo · 6.3 Um controlador
6.4 As views · 6.5 O item no menu · 6.6 Tutorial completo: CRUD de Professores

7. Configurações: o que dá para mudar

7.1 app.php · 7.2 banco.php · 7.3 Cores e visual
7.4 Menu · 7.5 Página inicial · 7.6 Campo novo em tabela existente

8. Escrevendo testes

9. Referência rápida

10. Problemas comuns

1. Colocando o sistema para rodar

1.1 O que você precisa ter instalado

Apenas o **XAMPP**. Ele já traz tudo: o PHP, o servidor Apache e o banco de dados MySQL/MariaDB. Não é preciso instalar Composer nem baixar bibliotecas — este framework não tem nenhuma dependência externa.

1.2 Instalação em 4 passos

- 1 Copie a pasta do projeto para dentro de `htdocs`**
No Windows, normalmente `C:\xampp\htdocs\phpprojeto`.
- 2 Abra o painel do XAMPP e clique em *Start* no Apache e no MySQL**
Os dois precisam ficar verdes. O Apache serve as páginas; o MySQL guarda os dados.
- 3 Acesse `http://localhost/phpprojeto/installar.php` uma única vez**
Esse arquivo cria o banco, as tabelas e cadastra 8 alunos de exemplo.
- 4 Acesse `http://localhost/phpprojeto`**
Pronto: o sistema está no ar.

O INSTALADOR FAZ TUDO SOZINHO

Você **não** precisa abrir o phpMyAdmin para criar o banco na mão. O `instalar.php` cria o banco `framework_aula`, executa o arquivo de estrutura das tabelas e insere os dados de exemplo. Pode rodá-lo quantas vezes quiser — o banco é sempre recriado do zero.

1.3 Comandos no terminal: use o caminho completo do PHP

LEIA ANTES DE DIGITAR QUALQUER COMANDO

O XAMPP **não** registra o PHP no Windows. Se você digitar só `php`, o terminal responde:

```
php : O termo 'php' nao e reconhecido como nome de cmdlet,
funcao, arquivo de script ou programa operavel.
```

Não é erro do projeto. Basta informar onde o PHP está — por isso **todos os comandos deste guia começam com** `C:\xampp\php\php.exe`.

Abra o PowerShell **na pasta do projeto** e rode:

PowerShell, dentro de `C:\xampp\htdocs\phpprojeto`

```
# cria o banco, as tabelas e os dados de exemplo
C:\xampp\php\php.exe instalar.php

# liga o servidor em http://localhost:8000
C:\xampp\php\php.exe -S localhost:8000 roteador.php
```

COMO ABRIR O TERMINAL JÁ NA PASTA CERTA

No Explorador de Arquivos, entre em `C:\xampp\htdocs\phpprojeto`, clique com o botão direito em um espaço vazio e escolha **“Abrir no Terminal”**. Assim você não precisa digitar o caminho do projeto em cada comando.

1.4 Conferindo se deu certo

Rode a bateria de testes automáticos. Ela verifica o framework inteiro em menos de um segundo:

```
C:\xampp\php\php.exe testes/executar.php
```

Se aparecer **“TUDO CERTO! O sistema esta funcionando.”**, está tudo em ordem.

CANSOU DE DIGITAR O CAMINHO?

Só nesta janela do terminal, crie um apelido — vale até fechá-la:

```
Set-Alias php C:\xampp\php\php.exe
```

A partir daí o `instalar.php` funciona normalmente *naquela janela*. Ao abrir um terminal novo, repita o comando.

1.5 Onde ficam as configurações

Arquivo	Para que serve
<code>configuracoes/app.php</code>	Nome do sistema, fuso horário, modo de depuração (<code>debug</code>) e qual controlador responde pela página inicial.
<code>configuracoes/banco.php</code>	Driver (<code>mysql</code> ou <code>sqlite</code>), host, nome do banco, usuário e senha.

Os valores padrão do XAMPP já vêm preenchidos: banco `framework_aula`, usuário `root` e senha vazia.

ENQUANTO ESTIVER DESENVOLVENDO

Deixe `'debug' => true` em `configuracoes/app.php`. Assim, quando houver um erro, a mensagem completa aparece na tela em vez de uma página genérica. Ao publicar o sistema de verdade, troque para `false`.

2. O que é MVC (e por que usar)

MVC é uma forma de **separar responsabilidades**. Em vez de um arquivo gigante que faz tudo — consulta o banco, decide as regras e ainda monta o HTML —, o código é dividido em três papéis:

Papel	Onde fica	Responsabilidade
Model (Modelo)	<code>modelos/</code>	Conversa com o banco de dados e guarda as regras de validação. É quem sabe <i>o que</i> é um aluno válido.
View (Visão)	<code>views/</code>	Monta o HTML que o usuário vê. Não decide nada, apenas exhibe o que recebeu.
Controller (Controlador)	<code>controllers/</code>	Recebe o pedido do navegador, chama o modelo e escolhe qual view mostrar. É o maestro.

Uma analogia

Pense em um restaurante:

- o **garçom** (Controller) recebe o pedido do cliente e leva para a cozinha;
- a **cozinha** (Model) tem os ingredientes e as receitas — ela sabe preparar o prato;
- o **prato montado** (View) é o que chega bonito à mesa.

O garçom não cozinha, e a cozinha não decide a decoração da mesa. Cada um faz a sua parte — e é por isso que dá para trocar o visual do sistema sem mexer em uma linha de SQL.

A REGRA DE OURO

SQL só existe dentro de `modelos/`. HTML só existe dentro de `views/`. Se você escreveu uma consulta SQL no meio de um controlador, ou um `<table>` dentro de um modelo, alguma coisa saiu do lugar.

E o que é “framework”?

É a estrutura pronta que fica na pasta `nucleo/`. Ela cuida do trabalho repetitivo — descobrir qual código responde por cada endereço, conectar no banco, montar o HTML dentro do template. Você **herda** dessas classes e ganha tudo isso de graça, escrevendo só o que é específico do seu sistema.

3. O caminho de uma requisição

Toda página do sistema segue exatamente o mesmo caminho. Entender esta sequência é entender o framework inteiro. Vamos acompanhar o que acontece quando alguém acessa

`http://localhost/phpprojeto/alunos/ver/7` :

- 1 O `.htaccess` intercepta o endereço**
Como não existe uma pasta chamada `alunos`, o Apache reescreve tudo para `index.php?url=alunos/ver/7`.
- 2 O `index.php` chama o `bootstrap.php`**
Define os caminhos das pastas, liga o autoloader, carrega as configurações e as funções de atalho, e inicia a sessão.
- 3 A classe `App` quebra a URL em três partes**
`alunos` vira o controlador, `ver` vira o método e `7` vira o parâmetro.
- 4 O `AlunosController::ver('7')` é executado**
O `App` confere se a classe existe, se o método é público e se a quantidade de parâmetros bate.
- 5 O controlador pede os dados ao modelo**
`$this->alunos->buscar(7)` devolve o registro do banco (ou `null`, e aí cai na tela 404).
- 6 O controlador manda desenhar a view**
`$this->view('alunos/ver', ['aluno' => $aluno])` executa `views/alunos/ver.php`.
- 7 A view é encaixada dentro do template**
O HTML gerado chega em `views/template/layout.php` pela variável `$conteudo`, já com cabeçalho e rodapé.
- 8 O HTML completo é enviado ao navegador**
Fim do ciclo.

Como a URL vira classe e método

Endereço acessado	Código que responde
<code>/</code>	<code>HomeController::index()</code> (o padrão, definido em <code>app.php</code>)
<code>/alunos</code>	<code>AlunosController::index()</code>
<code>/alunos/criar</code>	<code>AlunosController::criar()</code>
<code>/alunos/ver/7</code>	<code>AlunosController::ver('7')</code>
<code>/alunos/editar/7</code>	<code>AlunosController::editar('7')</code>
<code>/nota-final/por-turma</code>	<code>NotaFinalController::porTurma()</code>

Repare na última linha: o traço no endereço vira **letra maiúscula** no código. É a única “tradução” que o roteador faz.

SÓ MÉTODOS PÚBLICOS VIRAM ROTA

Métodos `private` e `protected` **não** podem ser acessados pela URL — o framework recusa com um erro 404. Use-os à vontade para o código de apoio do controlador, sem medo de que alguém os acesse pelo navegador.

4. Mapa das pastas

phpprojeto/ | — **index.php** porta de entrada: TODA página passa por aqui | — **instalar.php** cria o banco, as tabelas e os dados de exemplo | — **roteador.php** só para o servidor embutido do PHP | — **.htaccess** manda todas as URLs para o **index.php** | — **configuracoes/** ← você ajusta aqui | — **app.php** nome, debug, fuso, rota padrão | — **banco.php** driver, host, usuário, senha | — **controllers/** ← você cria aqui | — **HomeController.php** | — **AlunosController.php** CRUD completo: use de modelo | — **modelos/** ← você cria aqui | — **Aluno.php** tabela + regras de validação | — **views/** ← você cria aqui | — **alunos/** index, ver, formulario | — **home/** index, sobre | — **template/** layout, cabeçalho, rodapé, mensagens | — **erros/** telas de 404 e 500 | — **css/** estilo.css | — **javascripts/** app.js | — **nucleo/** o framework — normalmente você NÃO mexe | — **App.php** roteador | — **Controller.php** classe base dos controladores | — **Model.php** classe base dos modelos (CRUD pronto) | — **View.php** monta as telas | — **Database.php** conexão PDO | — **Validador.php** regras de formulário | — **Sql.php** proteção contra SQL Injection | — **Sessao.php** mensagens flash e sessão | — **Config.php** leitura das configurações | — **Autoloader.php** carrega as classes sozinho | — **helpers.php** funções de atalho: `e()`, `url()`, `asset()`... | — **bootstrap.php** prepara tudo antes de começar | — **testes/** ← você cria aqui | — **executar.php** roda todos os testes de uma vez | — **exemplos/** **ExemploTest.php**: comece por aqui | — **suporte/** o motor de testes | — **banco/** | — **esquema.mysql.sql** estrutura das tabelas (MySQL) | — **esquema.sqlite.sql** estrutura das tabelas (SQLite) | — **dados_exemplo.sql** registros iniciais

ONDE VOCÊ VAI TRABALHAR

Nas suas atividades você vai criar arquivos em **apenas quatro pastas**: **modelos/**, **controllers/**, **views/** e **testes/**. A pasta **nucleo/** é o motor pronto — mexa nela só quando o professor pedir.

Por que existe um **.htaccess** em cada pasta interna?

Segurança. As pastas **nucleo/**, **modelos/**, **controllers/**, **configuracoes/**, **testes/** e **banco/** têm cada uma o seu **.htaccess** bloqueando acesso direto pelo navegador. Sem isso, alguém poderia tentar abrir o arquivo com a sua senha do banco.

5. As peças do framework

Agora vamos abrir cada peça, na ordem em que elas entram em ação.

5.1 `index.php` — a porta de entrada

É o *front controller*: o único arquivo PHP que o navegador realmente abre. Todas as páginas passam por ele. São só três linhas úteis:

```
index.php

require_once __DIR__ . '/nucleo/bootstrap.php';

(new Nucleo\App())->executar();
```

Ter um único ponto de entrada é o que permite ter endereços limpos como `/alunos/ver/7` em vez de `/alunos_ver.php?id=7`.

5.2 `nucleo/bootstrap.php` — a preparação

Roda antes de qualquer coisa e deixa o ambiente pronto, nesta ordem:

1. **Define as constantes de caminho** — `CAMINHO_RAIZ`, `CAMINHO_VIEWS`, `CAMINHO_MODELOS` etc. Use-as sempre que precisar montar um caminho de arquivo.
2. **Liga o autoloader** e associa cada namespace à sua pasta.
3. **Carrega as configurações** e o arquivo de funções de atalho (`helpers.php`).
4. **Ajusta a exibição de erros** conforme o `debug`.
5. **Inicia a sessão**.

5.3 `Autoloader` — carregar classes sem `require`

Sem ele, você teria que escrever um `require` para cada classe que fosse usar. O autoloader guarda um mapa de *namespace* → *pasta*: quando o PHP encontra uma classe que ainda não conhece, ele procura o arquivo automaticamente.

Classe usada no código	Arquivo carregado
<code>Controllers\AlunosController</code>	<code>controllers/AlunosController.php</code>
<code>Modelos\Aluno</code>	<code>modelos/Aluno.php</code>
<code>Nucleo\Validador</code>	<code>nucleo/Validador.php</code>

CONSEQUÊNCIA PRÁTICA PARA VOCÊ

O **nome do arquivo** tem que ser **idêntico ao nome da classe**, respeitando maiúsculas e minúsculas.

A classe `Professor` precisa estar em `modelos/Professor.php` — `professor.php` não funciona.

5.4 Config — ler as configurações

Cada arquivo de `configuracoes/` vira um grupo. Navegue até o valor usando ponto:

```
Config::obter('app.nome');           // "Framework MVC - Curso Tecnico"
Config::obter('app.debug');           // true
Config::obter('banco.mysql.host');    // "localhost"
Config::obter('app.paginacao', 20);   // 20, se a chave nao existir
```

5.5 App — o roteador

É o coração do framework. Recebe a URL, descobre quem responde e executa. Antes de chamar o método, ele confere quatro coisas — e é daí que vêm as mensagens de erro mais comuns:

Verificação	Mensagem quando falha
A classe do controlador existe?	<i>Controlador nao encontrado</i>
O método existe?	<i>Metodo nao encontrado</i>
O método é público (e não estático)?	<i>Metodo nao acessivel</i>
Vieram parâmetros suficientes na URL?	<i>Faltam parametros</i>

5.6 Controller — a classe base dos controladores

Sua classe herda dela e já ganha todos estes atalhos prontos:

Método	O que faz
<code>\$this->view(\$tela, \$dados)</code>	Desenha a view dentro do template padrão.
<code>\$this->viewSemLayout(\$tela, \$dados)</code>	Desenha só a view, sem cabeçalho/rodapé (útil em AJAX).
<code>\$this->modelo('Aluno')</code>	Cria uma instância de um modelo da pasta <code>modelos/</code> .
<code>\$this->post('nome')</code>	Lê um campo do formulário, já com <code>trim()</code> .
<code>\$this->get('busca')</code>	Lê um valor da query string (<code>?busca=ana</code>).
<code>\$this->todosOsCampos()</code>	Devolve o <code>\$_POST</code> inteiro, com <code>trim()</code> em cada texto.
<code>\$this->ehPost()</code>	Informa se a requisição é o envio de um formulário.
<code>\$this->redirecionar('alunos')</code>	Manda o navegador para outra rota interna.
<code>\$this->mensagem('sucesso', '...')</code>	Guarda um aviso para aparecer na próxima tela.
<code>\$this->json(\$dados)</code>	Responde em JSON, para exercícios de API/AJAX.
<code>\$this->naoEncontrado('...')</code>	Interrompe e mostra a tela 404.

O padrão de um método que salva

Este é o modelo que você vai repetir em todos os cadastros — vale a pena decorar:

```
controllers/AlunosController.php
```

```

public function salvar(): void
{
    // 1. So aceita POST
    if (!$this->ehPost()) {
        $this->redirecionar('alunos/criar');
    }

    // 2. Le os dados e pede ao MODELO que valide
    $dados = $this->todosOsCampos();
    $erros = $this->alunos->validar($dados);

    // 3. Deu erro? Guarda o que foi digitado e volta ao formulario
    if ($erros !== []) {
        Sessao::guardarEntrada($dados);
        Sessao::guardarErros($erros);

        $this->mensagem('erro', 'Corrija os campos destacados.');
```

POR QUE REDIRECIONAR DEPOIS DE SALVAR?

Se você apenas mostrasse a tela, o usuário poderia apertar F5 e cadastrar o mesmo registro duas vezes. Redirecionando, o F5 apenas recarrega a página de exibição — sem gravar nada. Esse padrão tem nome: *POST-Redirect-GET*.

5.7 Model — a classe base dos modelos

Aqui está a maior economia de código do framework. Você declara quatro propriedades e ganha o CRUD inteiro por herança:

modelos/Aluno.php

```

class Aluno extends Model
{
    protected string $tabela      = 'alunos'; // obrigatorio
    protected string $chavePrimaria = 'id';
    protected array  $preenchiveis = ['nome', 'email', 'curso', 'nota'];
    protected string $ordemPadrao  = 'nome ASC';
}
```

Propriedade	Para que serve
<code>\$tabela</code>	Nome da tabela no banco. Obrigatório — sem ele o modelo nem é criado.
<code>\$chavePrimaria</code>	Coluna que identifica cada registro. Padrão: <code>id</code> .
<code>\$preencheveis</code>	Lista branca de colunas que podem ser gravadas. Protege contra o usuário enviar campos que não deveria.
<code>\$ordemPadrao</code>	Ordenação usada pelo <code>todos()</code> quando você não pede outra.

Métodos que você ganha de graça

Método	Devolve
<code>todos()</code> / <code>todos('nota DESC')</code>	Todos os registros, como array.
<code>buscar(\$id)</code>	Um registro, ou <code>null</code> se não existir.
<code>onde('curso', 'Informatica')</code>	Todos os registros que batem com a condição.
<code>primeiroOnde('email', \$email)</code>	Só o primeiro resultado, ou <code>null</code> .
<code>contar()</code>	Quantidade de registros (número inteiro).
<code>existe(\$id)</code>	<code>true</code> ou <code>false</code> .
<code>criar(\$dados)</code>	Insere e devolve o id gerado .
<code>atualizar(\$id, \$dados)</code>	<code>true</code> se alguma linha mudou.
<code>excluir(\$id)</code>	<code>true</code> se apagou.
<code>consultar(\$sql, \$params)</code>	SELECT livre — para as suas consultas próprias.
<code>executar(\$sql, \$params)</code>	INSERT/UPDATE/DELETE livre; devolve linhas afetadas.

Consultas próprias

Quando o CRUD pronto não basta, escreva o SQL usando `consultar()`. Os valores vão **sempre** como `?`, nunca dentro do texto:

```

public function aprovados(): array
{
    return $this->consultar('SELECT * FROM alunos WHERE nota >= 6 ORDER BY nota DESC');
}

public function porCurso(string $curso): array
{
    // O valor vai como parametro (?) - jamais concatenado.
    return $this->consultar('SELECT * FROM alunos WHERE curso = ?', [$curso]);
}

```

5.8 View e o template

A view é um arquivo PHP dentro de `views/` que contém principalmente HTML. As variáveis enviadas pelo controlador chegam prontas para uso:

```

// No controlador:
$this->view('alunos/ver', ['titulo' => 'Ana', 'aluno' => $aluno]);

// Em views/alunos/ver.php ja existem $titulo e $aluno.

```

O template `views/template/layout.php` envolve toda tela. A view renderizada chega nele pela variável `$conteudo`:

`views/template/layout.php` (trecho)

```

<body>
    <?= parcial('template/cabecalho') ?>

    <main class="conteudo">
        <?= parcial('template/mensagens') ?>
        <?= $conteudo ?>
    </main>

    <?= parcial('template/rodape') ?>
</body>

```

REGRA QUE NÃO SE NEGOCIA: SEMPRE ESCAPE COM E()

Ao imprimir qualquer dado vindo do usuário ou do banco, use `<?= e($valor) ?>`. Sem isso, alguém pode cadastrar um aluno chamado `<script>...</script>` e o código dele seria executado no navegador de quem abrir a lista. Isso se chama **XSS**, e a função `e()` é a defesa.

5.9 Validador — conferir formulários

As regras são encadeadas, uma atrás da outra. Cada campo guarda apenas a **primeira** mensagem de erro, para a tela não ficar poluída:

```

$validador = new Validador($dados);

$validador
->obrigatorio('nome', 'Nome')
->minimo('nome', 3, 'Nome')
->maximo('nome', 100, 'Nome')
->obrigatorio('email', 'E-mail')
->email('email', 'E-mail')
->dentroDe('curso', Aluno::CURSOS, 'Curso')
->numerico('nota', 'Nota')
->entre('nota', 0, 10, 'Nota');

return $validador->erros(); // array vazio = tudo certo

```

Regra	Verifica se...
<code>obrigatorio(\$campo)</code>	...foi preenchido.
<code>minimo(\$campo, 3)</code>	...tem pelo menos N caracteres.
<code>maximo(\$campo, 100)</code>	...não passa de N caracteres.
<code>email(\$campo)</code>	...é um e-mail válido.
<code>numerico(\$campo)</code>	...é um número.
<code>entre(\$campo, 0, 10)</code>	...está dentro da faixa.
<code>dentroDe(\$campo, \$lista)</code>	...é uma das opções aceitas.
<code>personalizada(\$campo, \$ok, \$msg)</code>	...passa numa condição escrita por você.

CAMPO VAZIO SÓ É COBRADO PELO OBRIGATORIO()

As demais regras ignoram valores vazios de propósito. Assim, um campo opcional como a nota pode ficar em branco sem gerar erro — mas, se for preenchido, precisa ser um número entre 0 e 10.

Regra que consulta o banco

Para “o e-mail não pode repetir”, use `personalizada()`. Repare no `$ignorarId`: sem ele, ao editar um aluno o sistema acusaria conflito do registro com ele mesmo.

```

$email = trim((string) ($dados['email'] ?? ''));

if ($email !== '' && $this->emailJaUsado($email, $ignorarId)) {
    $validador->personalizada('email', false, 'Este e-mail já está cadastrado.');
```


5.10 Sessão — mensagens que aparecem uma vez

Mensagem *flash* é aquela que aparece depois de salvar e some ao recarregar. O ciclo tem três etapas:

1 O controlador guarda e redireciona

```
$this->mensagem('sucesso', 'Aluno cadastrado!')
```

2 Na requisição seguinte, o framework retira a mensagem da sessão

Ela passa para a memória desta requisição e sai do `$_SESSION`.

3 A view exibe e a mensagem desaparece

O parcial `template/mensagens` já faz isso — você não precisa escrever nada.

Os tipos aceitos são `sucesso`, `erro`, `aviso` e `info` — cada um com a sua cor no CSS.

5.11 Database — a conexão

Cuida da conexão PDO e a reaproveita durante toda a requisição (padrão *Singleton*). Você raramente chama esta classe diretamente: o `Model` já faz isso por você.

Método	O que faz
<code>Database::conexao()</code>	Devolve o objeto PDO, criando-o na primeira chamada.
<code>Database::criarBancoSeNaoExistir()</code>	Cria o banco no MySQL, se ainda não existir.
<code>Database::migrar()</code>	Cria as tabelas lendo <code>banco/esquema.<driver>.sql</code> .
<code>Database::popular()</code>	Insere os registros de <code>banco/dados_exemplo.sql</code> .

O `instalar.php` é apenas a sequência dessas quatro chamadas.

5.12 Sql — proteção contra SQL Injection

Este é o assunto mais importante de segurança da disciplina. Existem **dois tipos** de coisa que entram num comando SQL, e cada um tem o seu tratamento:

Tipo	Exemplos	Como proteger
Valores	o nome digitado, a nota, o id	Vão como parâmetro <code>?</code> . O PDO envia o comando e os valores separadamente, então o que o usuário digitou nunca é interpretado como comando.
Identificadores	nome de tabela, de coluna, <code>ASC</code> / <code>DESC</code> , operadores	Não podem ser parâmetro — a linguagem SQL não permite. Por isso passam pela classe <code>Sql</code> , que só aceita nomes com letras, números e <code>_</code> , e operadores de uma lista fechada.

NUNCA FAÇA ISTO

```
// PERIGO: o que o usuario digitar vira comando SQL
$this->consultar("SELECT * FROM alunos WHERE nome = '$nome'");
```

Se alguém digitasse `' OR '1'='1` no campo de busca, veria a tabela inteira. Com um pouco mais de criatividade, apagaria o banco.

FAÇA ASSIM

```
$this->consultar('SELECT * FROM alunos WHERE nome = ?', [$nome]);
```

Buscas com LIKE

No `LIKE` existe um detalhe a mais: os curingas `%` e `_`. Se o usuário digitar `%` na busca, veria a tabela inteira. O método `Sql::comoLike()` neutraliza isso:

```
$termo = Sql::comoLike($termo);

return $this->consultar(
    'SELECT * FROM alunos WHERE nome LIKE ? ESCAPE ' . Sql::ESCAPE_LIKE,
    [$termo]
);
```

5.13 Funções de atalho (helpers)

Estão disponíveis em qualquer lugar: controladores, modelos e views.

Função	Para que serve
<code>e(\$texto)</code>	Escapa o texto antes de imprimir. Use sempre ao exibir dados do usuário.
<code>url('alunos/ver/7')</code>	Monta um link interno que funciona em qualquer pasta ou porta.
<code>asset('css/estilo.css')</code>	Monta o caminho de um arquivo estático dentro de <code>views/</code> .
<code>parcial('template/rodape')</code>	Inclui um pedaço de tela reaproveitável.
<code>antigo('nome')</code>	Repõe o valor digitado depois de um erro de validação.
<code>erro_de('email')</code>	Devolve a mensagem de erro daquele campo, ou <code>null</code> .
<code>tem_erro('email')</code>	<code>true</code> se o campo tem erro (para pintar a borda de vermelho).
<code>data_br('2026-08-14')</code>	Converte para <code>14/08/2026</code> .
<code>moeda_br(1234.5)</code>	Formata como <code>1.234,50</code> .
<code>dd(\$variavel)</code>	<i>Dump and die</i> : mostra o conteúdo e para tudo. A ferramenta de depuração mais usada.

SEMPRE USE URL() E ASSET()

Nunca escreva caminhos fixos como `/alunos` ou `/views/css/estilo.css`. Essas funções descobrem sozinhas a pasta onde o sistema está — é o que faz o mesmo código funcionar em `localhost:8000` e em `localhost/phpprojeto` sem alterar nada.

6. Como adicionar cada parte do sistema

Esta seção é a mais prática do guia. Primeiro cada peça isolada — tabela, modelo, controlador, view — e depois um tutorial que junta tudo.

6.1 Adicionar uma tabela no banco

As tabelas ficam em `banco/esquema.mysql.sql`. Cada uma começa com um `DROP TABLE IF EXISTS` — é isso que permite rodar o instalador quantas vezes quiser.

`banco/esquema.mysql.sql`

```
DROP TABLE IF EXISTS professores;

CREATE TABLE professores (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  nome        VARCHAR(100) NOT NULL,
  email       VARCHAR(150) NOT NULL UNIQUE,
  disciplina  VARCHAR(60)  NOT NULL,
  salario     DECIMAL(8,2)  NULL,
  ativo       TINYINT(1)  NOT NULL DEFAULT 1,
  criado_em   TIMESTAMP  NOT NULL DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_professores_disciplina (disciplina)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Depois **sempre** rode o `instalar.php` — é ele que executa esse arquivo.

Tipos de coluna que você vai usar

Tipo	Guarda	Quando usar
<code>INT</code>	número inteiro	Ids, quantidades, idade.
<code>VARCHAR(n)</code>	texto até n caracteres	Nome, e-mail, cidade. Use o tamanho real: <code>VARCHAR(100)</code> para nome.
<code>TEXT</code>	texto longo	Observações, descrição, comentário.
<code>DECIMAL(8,2)</code>	número com casas decimais	Dinheiro e notas. Não use <code>FLOAT</code> para dinheiro — ele arredonda errado.
<code>DATE</code>	data	Data de nascimento, vencimento.
<code>TIMESTAMP</code>	data e hora	Registro de quando algo foi criado.
<code>TINYINT(1)</code>	0 ou 1	Sim/não: ativo, pago, concluído.

O que cada palavra-chave faz

Palavra-chave	Efeito
<code>AUTO_INCREMENT PRIMARY KEY</code>	O banco gera o id sozinho, sempre único. Toda tabela precisa de um.
<code>NOT NULL</code>	A coluna não aceita ficar vazia.
<code>NULL</code>	A coluna pode ficar vazia (campo opcional).
<code>UNIQUE</code>	Não permite valores repetidos. Ideal para e-mail e CPF.
<code>DEFAULT valor</code>	Valor usado quando nada for informado.
<code>INDEX idx_...</code>	Acelera buscas naquela coluna. Vale a pena em colunas usadas em filtros.

Ligando duas tabelas (chave estrangeira)

Para dizer que cada aluno pertence a uma turma, guarde o id da turma no aluno:

```
CREATE TABLE turmas (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(60) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;  
  
CREATE TABLE alunos (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  turma_id INT NULL,  
  
  FOREIGN KEY (turma_id) REFERENCES turmas(id) ON DELETE SET NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

A ORDEM NO ARQUIVO IMPORTA

A tabela `turmas` precisa ser criada **antes** de `alunos`, senão o banco não encontra a referência. Regra prática: quem é apontado vem primeiro.

Para juntar as duas na consulta, use `JOIN` dentro do modelo:

```
public function comTurma(): array  
{  
  return $this->consultar(  
    'SELECT a.*, t.nome AS turma  
      FROM alunos a  
      LEFT JOIN turmas t ON t.id = a.turma_id  
      ORDER BY a.nome'  
  );  
}
```

6.2 Adicionar um modelo

Um arquivo em `modelos/`, com o nome **no singular** e igual ao da classe. O mínimo que funciona são cinco linhas:

`modelos/Professor.php`

```
<?php

namespace Modelos;

use Nucleo\Model;

class Professor extends Model
{
    protected string $tabela = 'professores';
    protected array $preencheveis = ['nome', 'email', 'disciplina'];
}
```

Pronto: `todos()`, `buscar()`, `criar()`, `atualizar()` e `excluir()` já funcionam. Daí em diante você só acrescenta o que for específico.

Checklist do modelo

✓	Item
1	Arquivo em <code>modelos/</code> com o mesmo nome da classe (<code>Professor.php</code> → <code>class Professor</code>).
2	Primeira linha: <code>namespace Modelos;</code>
3	<code>extends Model</code> (com <code>use Nucleo\Model;</code> no topo).
4	<code>\$tabela</code> preenchido — sem isso o modelo dispara erro ao ser criado.
5	<code>\$preencheveis</code> com todos os campos do formulário — o que não estiver aqui é ignorado ao salvar.

Acrescentando validação

Sobrescreva o método `validar()`. Ele recebe os dados do formulário e devolve um array de erros (vazio quer dizer que passou):

```

public function validar(array $dados, int|string|null $ignorarId = null): array
{
    $v = new Validador($dados);

    $v->obrigatorio('nome', 'Nome')
        ->minimo('nome', 3, 'Nome')
        ->obrigatorio('email', 'E-mail')
        ->email('email', 'E-mail');

    return $v->erros();
}

```

Acrescentando consultas próprias

Qualquer método público seu pode usar `consultar()`. Lembre: valores sempre como `?`.

```

public function ativos(): array
{
    return $this->consultar('SELECT * FROM professores WHERE ativo = 1 ORDER BY nome');
}

public function porDisciplina(string $disciplina): array
{
    return $this->consultar(
        'SELECT * FROM professores WHERE disciplina = ? ORDER BY nome',
        [$disciplina]
    );
}

```

6.3 Adicionar um controlador

Um arquivo em `controllers/`. O nome vai **no plural** e **obrigatoriamente** termina em `Controller` — é assim que o roteador o encontra.

```
controllers/ProfessoresController.php
```

```

<?php

namespace Controllers;

use Modelos\Professor;
use Nucleo\Controller;

class ProfessoresController extends Controller
{
    private Professor $professores;

    public function __construct()
    {
        $this->professores = new Professor();
    }

    public function index(): void
    {
        $this->view('professores/index', [
            'titulo'      => 'Professores',
            'professores' => $this->professores->todos(),
        ]);
    }
}

```

Como o nome do arquivo vira endereço

Arquivo	Endereço que passa a existir
ProfessoresController.php	/professores → método index()
método criar()	/professores/criar
método ver(string \$id)	/professores/ver/5
método porTurma()	/professores/por-turma

ERROS QUE APARECEM TODA HORA

“Controlador não encontrado” — falta o sufixo `Controller` no nome da classe, ou o arquivo tem nome diferente da classe.

“Método não acessível” — o método está `private` ou `protected`; para virar rota tem que ser `public`.

“Faltam parametros” — o método pede um parâmetro obrigatório que não veio na URL. Ou passe o valor, ou dê um padrão: `ver(string $id = '1')`.

Os cinco métodos de um CRUD

Método	Endereço	Faz
<code>index()</code>	<code>/professores</code>	Lista tudo.
<code>criar()</code>	<code>/professores/criar</code>	Mostra o formulário vazio.
<code>salvar()</code>	<code>/professores/salvar</code>	Recebe o POST, valida e grava.
<code>editar(\$id)</code>	<code>/professores/editar/5</code>	Mostra o formulário preenchido.
<code>atualizar(\$id)</code>	<code>/professores/atualizar/5</code>	Recebe o POST e salva a edição.
<code>excluir(\$id)</code>	<code>/professores/excluir/5</code>	Apaga o registro.

6.4 Adicionar as views

Crie uma pasta em `views/` com o nome do recurso **no plural**. Dentro dela, um arquivo `.php` por tela.

```
views/professores/ └─ index.php ← $this->view('professores/index') └─ ver.php ← $this->view('professores/ver') └─ formulario.php ← $this->view('professores/formulario')
```

O caminho passado no controlador é o caminho do arquivo sem o `.php`. A view recebe exatamente as variáveis que o controlador enviou:

```
// No controlador
$this->view('professores/index', [
    'titulo'      => 'Professores',    // vira $titulo
    'professores' => $lista,           // vira $professores
]);
```

ESCAPE SEMPRE

Todo dado que vem do banco ou do usuário sai pela função `e()`: `<?= e($p['nome']) ?>`. Sem isso o sistema fica vulnerável a XSS.

Estrutura típica de uma listagem

```
<h1>Professores</h1>

<a class="botao" href="<?= url('professores/criar') ?>">Novo professor</a>

<!-- Lista vazia: avise, não mostre uma tabela em branco -->
<?php if ($professores === []): ?>
    <p>Nenhum professor cadastrado ainda.</p>
<?php else: ?>
    <table>
        <thead>
            <tr><th>Nome</th><th>Disciplina</th><th>Acoes</th></tr>
        </thead>
        <tbody>
            <?php foreach ($professores as $p): ?>
                <tr>
                    <td><?= e($p['nome']) ?></td>
                    <td><?= e($p['disciplina']) ?></td>
                    <td>
                        <a href="<?= url('professores/editar/' . $p['id']) ?>">Editar</a>
                    </td>
                </tr>
            <?php endforeach ?>
        </tbody>
    </table>
<?php endif ?>
```

POR QUE IF: ... ENDIF EM VEZ DE CHAVES?

Nas views, essa forma deixa o HTML muito mais legível — dá para ver onde cada bloco termina sem procurar a chave correspondente no meio das tags. Nos controladores e modelos, use chaves normalmente.

6.5 Adicionar o item no menu

O menu fica em `views/template/cabecalho.php`, num array simples. Acrescente uma linha:

`views/template/cabecalho.php`

```
$itens = [
    'home'      => ['rota' => '',          'texto' => 'Inicio'],
    'alunos'    => ['rota' => 'alunos',    'texto' => 'Alunos'],
    'professores' => ['rota' => 'professores', 'texto' => 'Professores'],
];
```

A **chave** do array (`'professores'`) precisa ser igual ao primeiro pedaço da URL — é ela que faz o item ficar destacado quando você está naquela seção.

6.6 Tutorial completo: CRUD de Professores

Agora juntando tudo, do banco até a tela. São cinco passos — ao final você terá repetido tudo o que o `AlunosController` faz.

Passo 1 — Criar a tabela

Acrescente ao final de `banco/esquema.mysql.sql`:

`banco/esquema.mysql.sql`

```
DROP TABLE IF EXISTS professores;

CREATE TABLE professores (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    nome        VARCHAR(100) NOT NULL,
    disciplina   VARCHAR(60)  NOT NULL,
    criado_em    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Depois rode o `instalar.php` de novo para o banco ser recriado com a tabela nova.

Passo 2 — Criar o modelo

`modelos/Professor.php`

```
<?php

namespace Modelos;

use Nucleo\Model;
use Nucleo\Validator;

class Professor extends Model
{
    protected string $tabela      = 'professores';
    protected array  $preencheis  = ['nome', 'disciplina'];
    protected string $ordemPadrao = 'nome ASC';

    public function validar(array $dados, int|string|null $ignorarId = null): array
    {
        $v = new Validator($dados);

        $v->obrigatorio('nome', 'Nome')
            ->minimo('nome', 3, 'Nome')
            ->obrigatorio('disciplina', 'Disciplina');

        return $v->erros();
    }
}
```

Passo 3 — Criar o controlador

```
controllers/ProfessoresController.php
```

```

<?php

namespace Controllers;

use Modelos\Professor;
use Nucleo\Controller;
use Nucleo\Sessao;

class ProfessoresController extends Controller
{
    private Professor $professores;

    public function __construct()
    {
        $this->professores = new Professor();
    }

    // /professores
    public function index(): void
    {
        $this->view('professores/index', [
            'titulo'      => 'Professores',
            'professores' => $this->professores->todos(),
        ]);
    }

    // /professores/criar
    public function criar(): void
    {
        $this->view('professores/formulario', [
            'titulo' => 'Novo professor',
            'acao'   => url('professores/salvar'),
        ]);
    }

    // /professores/salvar (POST)
    public function salvar(): void
    {
        if (!$this->ehPost()) {
            $this->redirecionar('professores/criar');
        }

        $dados = $this->todosOsCampos();
        $erros = $this->professores->validar($dados);

        if ($erros !== []) {
            Sessao::guardarEntrada($dados);
            Sessao::guardarErros($erros);

            $this->mensagem('erro', 'Corrija os campos destacados.');
```

```

        $this->redirecionar('professores');
    }

    // /professores/excluir/5
    public function excluir(string $id): void
    {
        if (!$this->professores->existe($id)) {
            $this->naoEncontrado("Professor {$id} nao existe.");
        }

        $this->professores->excluir($id);

        $this->mensagem('sucesso', 'Professor removido.');
```

```

        $this->redirecionar('professores');
    }
}

```

Passo 4 — Criar as views

Crie a pasta `views/professores/` com dois arquivos.

`views/professores/index.php`

```

<h1>Professores</h1>

<a class="botao" href="<?= url('professores/criar') ?>">Novo professor</a>

<table>
    <thead>
        <tr><th>Nome</th><th>Disciplina</th><th></th></tr>
    </thead>
    <tbody>
        <?php foreach ($professores as $p): ?>
            <tr>
                <td><?= e($p['nome']) ?></td>
                <td><?= e($p['disciplina']) ?></td>
                <td>
                    <a href="<?= url('professores/excluir/' . $p['id']) ?>">Excluir</a>
                </td>
            </tr>
        <?php endforeach ?>
    </tbody>
</table>

```

`views/professores/formulario.php`

```

<h1>Novo professor</h1>

<form method="POST" action="<?= e($acao) ?>" class="formulario">

    <div class="campo <?= tem_erro('nome') ? 'campo--erro' : '' ?>">
        <label for="nome">Nome</label>
        <input type="text" id="nome" name="nome"
            value="<?= e(antigo('nome')) ?>" required>

        <?php if ($msg = erro_de('nome')): ?>
            <span class="campo__erro"><?= e($msg) ?></span>
        <?php endif ?>
    </div>

    <div class="campo <?= tem_erro('disciplina') ? 'campo--erro' : '' ?>">
        <label for="disciplina">Disciplina</label>
        <input type="text" id="disciplina" name="disciplina"
            value="<?= e(antigo('disciplina')) ?>" required>

        <?php if ($msg = erro_de('disciplina')): ?>
            <span class="campo__erro"><?= e($msg) ?></span>
        <?php endif ?>
    </div>

    <button type="submit" class="botao">Cadastrar</button>
</form>

```

Passo 5 — Testar

Acesse `http://localhost/phpprojeto/professores`. Sem escrever uma linha de configuração de rota, o endereço já funciona — o roteamento é por convenção.

CONFIRA O QUE VOCÊ FEZ

Repare que em nenhum momento você escreveu `INSERT`, `SELECT` ou `DELETE`. Tudo veio por herança de `Nucleo\Model`. É exatamente para isso que serve um framework.

7. Configurações: o que dá para mudar

Quase tudo que muda de um sistema para outro está concentrado em poucos arquivos. Esta seção lista cada ajuste e onde ele é feito.

7.1 `configuracoes/app.php` — a aplicação

Chave	Padrão	O que muda
<code>nome</code>	<i>Framework MVC...</i>	Nome exibido no cabeçalho e no título da aba do navegador.
<code>url_base</code>	<code>''</code> (vazio)	Endereço raiz. Vazio faz o framework detectar sozinho — funciona tanto em <code>localhost:8000</code> quanto em <code>localhost/phpprojeto</code> . Deixe vazio.
<code>debug</code>	<code>true</code>	<code>true</code> mostra o erro completo na tela; <code>false</code> mostra uma tela genérica. Use <code>false</code> só ao publicar.
<code>timezone</code>	<code>America/Sao_Paulo</code>	Fuso usado nas datas.
<code>controlador_padrao</code>	<code>home</code>	Quem responde pelo endereço <code>/</code> .
<code>metodo_padrao</code>	<code>index</code>	Método chamado quando a URL não indica um.

Trocando o nome do sistema

```
return [  
    'nome' => 'Secretaria Escolar', // aparece no cabeçalho e no <title>  
    // ...  
];
```

Criando uma configuração sua

Qualquer chave que você acrescentar fica disponível na hora, sem mexer em mais nada:

```
// Em configuracoes/app.php  
'itens_por_pagina' => 20,  
  
// Em qualquer lugar do sistema  
$porPagina = Config::obter('app.itens_por_pagina', 10);
```

Você também pode criar um arquivo novo em `configuracoes/`. O arquivo `escola.php` vira o grupo `escola`, lido com `Config::obter('escola.chave')`.

7.2 configuracoes/banco.php — o banco de dados

configuracoes/banco.php

```
return [
    // 'sqlite' ou 'mysql'
    'driver' => 'mysql',

    'sqlite' => [
        'arquivo' => CAMINHO_RAIZ . '/banco/dados.sqlite',
    ],

    'mysql' => [
        'host'     => 'localhost',
        'porta'    => 3306,
        'banco'    => 'framework_aula',
        'usuario'  => 'root',
        'senha'    => '',
        'charset'  => 'utf8mb4',
    ],
];
```

Quero...	Faça
Trocar o nome do banco	Altere <code>'banco'</code> e rode o <code>instalar.php</code> . O banco novo é criado sozinho.
Usar senha no MySQL	Preencha <code>'senha'</code> com a mesma senha configurada no XAMPP.
Rodar sem servidor de banco	Troque <code>'driver'</code> para <code>'sqlite'</code> e rode o instalador. Vira um único arquivo em <code>banco/dados.sqlite</code> .
Mudar a porta	Altere <code>'porta'</code> (o XAMPP às vezes usa 3307 quando 3306 está ocupada).

AO TROCAR O DRIVER, LEMBRE DO ESQUEMA

Cada driver lê o seu próprio arquivo de estrutura: `banco/esquema.mysql.sql` ou `banco/esquema.sqlite.sql`. Se você criou uma tabela nova só no arquivo do MySQL, ela não vai existir ao mudar para SQLite. Mantenha os dois em dia se for usar os dois.

7.3 Cores e visual

Todo o visual sai de um punhado de variáveis no topo de `views/css/estilo.css`. Mude ali e o sistema inteiro acompanha:

views/css/estilo.css

```

:root {
  --cor-fundo:      #f4f6fb; /* fundo da pagina */
  --cor-superficie: #ffffff; /* fundo dos cartoes e do cabecalho */
  --cor-borda:      #dfe3ec;
  --cor-texto:      #1f2430;
  --cor-texto-suave: #5c6478;
  --cor-primaria:   #2f6df6; /* botoes e links */
  --cor-primaria-esc: #1f4fc4; /* botao com o mouse em cima */
  --cor-sucesso:    #1e8f5a; /* mensagem de sucesso */
  --cor-erro:       #d13c3c; /* mensagem de erro */
  --cor-aviso:      #b8860b;
  --raio:           10px; /* arredondamento dos cantos */
  --sombra:         0 1px 3px rgba(23, 30, 51, .08);
  --largura-maxima: 1040px; /* largura do conteudo */
}

```

Para deixar o sistema com a identidade verde da escola, por exemplo, bastam duas linhas:

```

--cor-primaria:    #1e8f5a;
--cor-primaria-esc: #166b43;

```

ONDE FICAM OS OUTROS ARQUIVOS DE FRONT-END

`views/css/estilo.css` — todo o estilo · `views/javascripts/app.js` — o JavaScript ·
`views/imagens/` — as imagens. Todos são carregados pelo template com `asset()`.

7.4 Cabeçalho, menu e rodapé

Arquivo	Controla
<code>views/template/layout.php</code>	A estrutura geral da página: <code><head></code> , onde entram o CSS e o JS, e a ordem cabeçalho → conteúdo → rodapé.
<code>views/template/cabecalho.php</code>	A logo e o menu de navegação.
<code>views/template/rodape.php</code>	O rodapé.
<code>views/template/mensagens.php</code>	Como as mensagens de sucesso e erro aparecem.

Para acrescentar um item ao menu, veja a seção **6.5**.

7.5 Trocar a página inicial

Quem responde pelo endereço `/` é definido em `configuracoes/app.php`. Para que a lista de alunos passe a ser a primeira tela:

```
'controlador_padrao' => 'alunos', // AlunosController
'metodo_padrao'      => 'index',  // ::index()
```

Se quiser apenas mudar o *conteúdo* da página inicial mantendo o `HomeController`, edite `views/home/index.php`.

7.6 Adicionar um campo a uma tabela que já existe

Este é o ajuste mais comum durante as atividades — e o que mais gera dúvida, porque exige mexer em **quatro** lugares. Vamos acrescentar `telefone` aos alunos:

- 1 A coluna, em `banco/esquema.mysql.sql`**
Acrescente `telefone VARCHAR(20) NULL,` dentro do `CREATE TABLE alunos` e rode o `instalar.php`.
- 2 A permissão, em `modelos/Aluno.php`**
Inclua `'telefone'` em `$preencheis` — sem isso o campo é silenciosamente descartado ao salvar.
- 3 A regra, no `validar()` do modelo**
Ex.: `->maximo('telefone', 20, 'Telefone')`. Opcional, mas recomendado.
- 4 O campo, em `views/alunos/formulario.php`**
Copie um bloco `<div class="campo">` existente e troque o nome. Para exibir, acrescente também em `ver.php` e `index.php`.

O ESQUECIMENTO CLÁSSICO

Adicionar o campo no formulário e na tabela, mas **esquecer do `$preencheis`**. O sistema não dá erro nenhum: ele simplesmente salva o registro sem aquele campo, e você fica procurando o problema no lugar errado. Se um campo “não está salvando”, confira o `$preencheis` primeiro.

O bloco de campo para copiar

```
<div class="campo" <?= tem_erro('telefone') ? 'campo--erro' : '' ?>>
  <label for="telefone">Telefone</label>
  <input type="text" id="telefone" name="telefone"
    value="<?= e(antigo('telefone', $aluno['telefone'] ?? '')) ?>">

  <?php if ($msg = erro_de('telefone')): ?>
    <span class="campo__erro"><?= e($msg) ?></span>
  <?php endif ?>
</div>
```

RODAR O INSTALADOR APAGA OS DADOS

O `instalar.php` recria as tabelas do zero (`DROP TABLE`), então tudo que você cadastrou pelo sistema se perde. Durante as aulas isso não é problema — os dados de exemplo voltam automaticamente. Se precisar preservar registros, acrescente a coluna direto pelo phpMyAdmin com `ALTER TABLE alunos ADD telefone VARCHAR(20) NULL;` e atualize o arquivo de esquema para a próxima instalação.

7.7 Resumo: onde mexer para cada mudança

Quero mudar...	Arquivo
O nome do sistema	<code>configuracoes/app.php</code> → <code>nome</code>
As cores	<code>views/css/estilo.css</code> → bloco <code>:root</code>
A página inicial	<code>configuracoes/app.php</code> → <code>controlador_padrao</code>
O menu	<code>views/template/cabecalho.php</code> → array <code>\$itens</code>
O rodapé	<code>views/template/rodape.php</code>
Usuário/senha do banco	<code>configuracoes/banco.php</code> → <code>mysql</code>
As tabelas	<code>banco/esquema.mysql.sql</code> , depois rodar o <code>instalar.php</code>
Os dados de exemplo	<code>banco/dados_exemplo.sql</code>
As regras de validação	<code>modelos/<Nome>.php</code> → método <code>validar()</code>
Quais campos podem ser salvos	<code>modelos/<Nome>.php</code> → <code>\$preencheveis</code>
Ver o erro completo na tela	<code>configuracoes/app.php</code> → <code>debug => true</code>

8. Escrevendo testes

Teste automático é um código que verifica se outro código está certo. Em vez de abrir o navegador e conferir tudo na mão a cada alteração, você roda um comando e o computador confere por você:

```
C:\xampp\php\php.exe testes/executar.php
```

OS TESTES NUNCA ESTRAGAM OS SEUS DADOS

Eles usam um **SQLite em memória**, criado do zero a cada execução e apagado quando o processo termina. Pode rodar quantas vezes quiser: o banco MySQL do sistema não é tocado.

As três regras

1. O arquivo termina em `Test.php` e fica em uma subpasta de `testes/`.
2. A classe tem o mesmo nome do arquivo e herda de `TesteBase`.
3. Cada método público que começa com `teste` é executado automaticamente.

Um teste completo

```
testes/modelos/ProfessorTest.php
```

```

<?php

namespace Testes\Modelos;

use Modelos\Professor;
use Testes\Suporte\TesteBase;

class ProfessorTest extends TesteBase
{
    private Professor $professores;

    // Roda ANTES de cada teste: garante que cada um começa do zero.
    public function preparar(): void
    {
        $this->limparTabela('professores');
        $this->professores = new Professor();
    }

    public function testeCadastraUmProfessor(): void
    {
        $id = $this->professores->criar([
            'nome'      => 'Marina Alves',
            'disciplina' => 'Banco de Dados',
        ]);

        $this->assertVerdadeiro($id > 0);
        $this->assertIgual(1, $this->contarNaTabela('professores'));
    }

    public function testeNomeCurtoNaoPassa(): void
    {
        $erros = $this->professores->validar([
            'nome'      => 'Jo',
            'disciplina' => 'Redes',
        ]);

        $this->assertTemChave('nome', $erros);
    }

    // Teste de integracao: simula um acesso do navegador.
    public function testeListagemAbre(): void
    {
        $resposta = $this->requisitar('professores');

        $this->assertContem('Professores', $resposta->html());
    }
}

```

Verificações disponíveis

Método	Passa quando...
<code>assertIgual(\$esperado, \$obtido)</code>	...os valores são iguais (<code>5 == '5'</code> passa).
<code>assertIdentico(\$esperado, \$obtido)</code>	...são iguais e do mesmo tipo.
<code>assertDiferente(\$nao, \$obtido)</code>	...são diferentes.
<code>assertVerdadeiro(\$x)</code> / <code>assertFalso(\$x)</code>	...é exatamente <code>true</code> / <code>false</code> .
<code>assertNulo(\$x)</code> / <code>assertNaoNulo(\$x)</code>	...é <code>null</code> / não é <code>null</code> .
<code>assertVazio(\$x)</code> / <code>assertNaoVazio(\$x)</code>	...está vazio / tem conteúdo.
<code>assertContem(\$trecho, \$texto)</code>	...o texto contém o trecho.
<code>assertTotal(3, \$array)</code>	...o array tem exatamente N itens.
<code>assertTemChave('nome', \$array)</code>	...o array possui aquela chave.
<code>assertTemValor(\$v, \$array)</code>	...o valor está dentro do array.
<code>assertExcecao(Classe::class, fn)</code>	...a função dispara aquela exceção.

Ajudantes de teste

Método	O que faz
<code>\$this->requisitar('alunos')</code>	Simula um acesso GET e devolve a resposta.
<code>\$this->postar('alunos/salvar', \$dados)</code>	Simula o envio de um formulário.
<code>\$this->limparTabela('alunos')</code>	Esvazia a tabela — use no <code>preparar()</code> .
<code>\$this->contarNaTabela('alunos')</code>	Conta as linhas direto no banco, sem passar pelo modelo.
<code>\$this->limparSessao()</code>	Zera mensagens e dados de formulário.

COMECE PELO EXEMPLO

O arquivo `testes/exemplos/ExemploTest.php` não testa o sistema: ele demonstra cada verificação disponível, comentada. Use-o como consulta rápida enquanto escreve os seus.

9. Referência rápida

Comandos

Todos rodam no PowerShell aberto **dentro da pasta do projeto**. O caminho completo é obrigatório porque o XAMPP não registra o PHP no Windows.

Comando	O que faz
<code>C:\xampp\php\php.exe instalar.php</code>	Cria o banco, as tabelas e os dados de exemplo.
<code>C:\xampp\php\php.exe testes/executar.php</code>	Roda todos os testes automáticos.
<code>C:\xampp\php\php.exe testes/executar.php Modelos</code>	Roda só os testes da pasta <code>modelos</code> .
<code>C:\xampp\php\php.exe -S localhost:8000 roteador.php</code>	Liga o servidor embutido do PHP.
<code>Set-Alias php C:\xampp\php\php.exe</code>	Cria o apelido <code>php</code> — vale só na janela atual do terminal.

Endereços do sistema de exemplo

Endereço	Tela
<code>/</code>	Página inicial
<code>/alunos</code>	Lista de alunos (com busca)
<code>/alunos/ver/1</code>	Detalhe de um aluno
<code>/alunos/criar</code>	Formulário de cadastro
<code>/alunos/editar/1</code>	Formulário de edição
<code>/alunos/excluir/1</code>	Remove o registro
<code>/alunos/api</code>	Resposta em JSON (exemplo de API)
<code>/home/sobre</code>	Página “sobre”

Checklist ao criar um recurso novo

✓	Arquivo	Detalhe que costuma ser esquecido
1	<code>banco/esquema.mysql.sql</code>	Rodar o <code>instalar.php</code> depois de alterar.
2	<code>modelos/Nome.php</code>	Declarar <code>\$tabela</code> e <code>\$preencheis</code> .
3	<code>controllers/NomesController.php</code>	A classe precisa terminar em <code>Controller</code> .
4	<code>views/nomes/index.php</code>	Escapar tudo com <code>e()</code> .
5	<code>testes/modelos/NomeTest.php</code>	Limpar a tabela no <code>preparar()</code> .

Convenções de nome

Item	Padrão	Exemplo
Tabela	plural, minúsculo	<code>professores</code>
Modelo	singular, PascalCase	<code>Professor</code>
Controlador	plural + <code>Controller</code>	<code>ProfessoresController</code>
Pasta de views	plural, minúsculo	<code>views/professores/</code>
Teste	nome + <code>Test</code>	<code>ProfessorTest</code>

10. Problemas comuns

Sintoma	Causa provável e solução
"O termo 'php' não é reconhecido..."	O XAMPP não registra o PHP no Windows. Use o caminho completo: <code>C:\xampp\php\php.exe testes/executar.php</code> (veja a seção 1.3).
"Could not open input file: instalar.php"	O terminal está em outra pasta. Entre em <code>C:\xampp\htdocs\phpprojeto</code> antes de rodar o comando.
"Não foi possível conectar ao banco"	O MySQL não está ligado. Abra o painel do XAMPP e clique em <i>Start</i> no MySQL.
"Base table or view not found"	As tabelas ainda não existem. Rode o <code>instalar.php</code> .
"Access denied for user 'root'"	Usuário ou senha errados em <code>configuracoes/banco.php</code> .
Todas as rotas dão 404 no XAMPP	<code>mod_rewrite</code> desativado ou <code>AllowOverride None</code> no Apache.
"Controlador nao encontrado"	O nome da classe precisa terminar em <code>Controller</code> e ser idêntico ao nome do arquivo.
"Metodo nao acessivel"	O método precisa ser <code>public</code> para virar rota.
"Faltam parametros"	O método exige um parâmetro que não veio na URL. Ex.: <code>/alunos/ver</code> sem o id.
"View nao encontrada"	Confira o caminho: <code>alunos/index</code> corresponde a <code>views/alunos/index.php</code> .
"Defina a propriedade \$tabela"	O modelo foi criado sem declarar <code>protected string \$tabela</code> .
"Nome de coluna invalido"	É a proteção contra SQL Injection agindo: nomes de coluna só aceitam letras, números e <code>_</code> .
"Nenhum campo valido para inserir"	Os campos do formulário não estão listados em <code>\$preenchiveis</code> .
Um campo simplesmente não salva (sem erro nenhum)	Falta incluir esse campo em <code>\$preenchiveis</code> , ou o <code>name=""</code> do input está diferente do nome da coluna.
"Unknown column" ao salvar	A coluna existe no modelo mas não na tabela. Atualize <code>banco/esquema.mysql.sql</code> e rode o <code>instalar.php</code> .
Item do menu não fica destacado	A chave do array <code>\$itens</code> precisa ser igual ao primeiro pedaço da URL.
CSS não carrega	Use sempre <code>asset('css/estilo.css')</code> , nunca um caminho fixo.

Sintoma	Causa provável e solução
Página em branco	Coloque <code>'debug' => true</code> em <code>configuracoes/app.php</code> para ver o erro.

Como investigar qualquer erro

1

Leia a mensagem inteira

Com `debug` ligado, ela diz o arquivo e a linha exata.

2

Use `dd()` para ver o que tem dentro da variável

Coloque `dd($dados);` antes da linha problemática e recarregue a página.

3

Rode os testes

`C:\xampp\php\php.exe testes/executar.php` mostra rapidamente se você quebrou algo que já funcionava.

ÚLTIMA DICA

Quando ficar em dúvida sobre como fazer alguma coisa, abra o `AlunosController.php` e o `Aluno.php`. Eles foram escritos para servir de modelo: praticamente tudo o que você precisa fazer nas atividades já está resolvido lá, com comentários explicando o porquê de cada escolha.